ESTUDO COMPARATIVO DE ALGORITMOS DE APRENDIZAGEM DE MÁQUINA NAS BASES TITANIC E BREAST CANCER

Eduardo Antonio da Silva (12311BSI317)

Rogério Anastácio (12311BSI279)

Orientadora: Prof. Dra. Elaine Ribeiro Faria

1. Introdução

Na atualidade os dados são coletados em grande volume pelos sistemas computacionais em todo o mundo. Seja através do uso corriqueiro da internet acessando *websites* ou com o uso mais sofisticado realizando transações bancárias ou na bolsa de valores. Pode-se dizer que, "(...) *atualmente, em cada 10 minutos são gerados mais dados do que todos os dados gerados desde a pré-história até o ano de 2003.*" (CAVIQUE, 2014, p.11).

Os dados na forma pura (sem processamento) não possuem valor para as pessoas e empresas, pois não dizem nada de forma isolada. E como o volume de dados coletados existe em uma quantidade absurdamente grande, a tarefa de análise torna-se humanamente impossível de ser realizada sem a aplicação de técnicas e processos computacionais.

Assim para que esses dados possam agregar valor, é necessário tratá-los com métodos de pré-processamento para que, então, seja possível aplicar métricas estatísticas e matemáticas nos dados, com a finalidade de extrair informação útil e a partir disso analisados pelos interessados.

Dentre as metodologias de análise disponíveis, destacam-se os algoritmos de Aprendizado de Máquina (*Machine Learning*), que permitem aos computadores aprenderem padrões a partir de dados históricos para realizar predições em novos dados. No contexto de classificação supervisionada, o objetivo é ensinar o modelo a distinguir entre diferentes categorias, como prever se um passageiro sobreviveria a um naufrágio ou se um tumor é maligno.

Contudo, a eficácia desses modelos depende intrinsecamente da natureza dos dados e do pré-processamento aplicado. Diferentes algoritmos, como Árvores de Decisão ou Vizinhos Próximos (KNN), possuem vieses e complexidades distintas que afetam seu desempenho. Portanto, torna-se essencial não apenas aplicar um algoritmo, mas comparar diferentes abordagens para identificar qual oferece a melhor generalização para o problema em questão.

1.1. Objetivos

O objetivo desse trabalho é fazer um estudo comparativo em algoritmos de aprendizagem de máquina supervisionada. E para isso serão aplicadas técnicas de mineração

de dados em duas bases de dados diferentes para avaliar e interpretar o desempenho dessas técnicas.

2. Bases de dados utilizadas

Foram selecionadas duas bases de dados distintas, armazenadas em repositório público com tipos de dados díspares. Essas coleções de dados, com todas as suas características, são apresentadas na sequência.

2.1. *Titanic - Dataset*

Essa base de dados contém algumas informações conhecidas a respeito das pessoas que embarcaram na viagem inaugural do navio *RMS Titanic* em 1912 (RMS TITANIC, 2025). Ela contém 891 registros com doze colunas com valores numéricos e alfanuméricos e alguns valores ausentes. A base de dados "*Titanic - Dataset*" (ALI, [s.d.]) pode ser acessada e copiada através do site da comunidade *Kaggle*.

Na tabela 1, abaixo, é apresentada o nome de cada coluna, o tipo de dado e se possui valor ausente nessa base de dados.

Tabela 1 – Atributos da base da dados "*Titanic – Machine Learning from Disaster*".

Nome Variável	Tradução / Significado	Tipo do Atributo	Valores Ausentes (S/N)
PassengerId	Número Identificador do passageiro	Identificador	Não
Survived	Sobreviveu	Categórico Nominal Binário	Não
Pclass	Classe de Embarque	Categórico Ordinal	Não
Name	Nome	Categórico Nominal	Não
Sex	Gênero	Categórico Nominal Binário	Não
Age	Idade	Numérico	Sim
SibSp	Número Irmãos Embarcados	Numérico	Não
Parch	Número Pais Embarcados	Numérico	Não
Ticket	Número do Ticket	Categórico Nominal	Não
Fare	Tarifa do Ticket	Numérico	Não
Cabin	Número da Cabine	Categórico Nominal	Sim
Embarked	Cidade Embarque	Categórico Nominal	Sim

2.2. *Breast Cancer Wisconsin (Diagnostic)*

O conjunto de dados "*Breast Cancer Wisconsin (Diagnostic)"* (WOLBERG, 1995) são atributos que foram extraídos de imagens digitalizadas de lâminas a partir de uma punção aspirativa de uma massa mamária que descrevem características dos núcleos celulares.

Nessa classe são utilizados 32 atributos: raio, textura, perímetro, área, suavidade, concavidade, simetria e dimensão fractal. Nela tem 569 instâncias, sendo que 357 são amostras benignas e 212 amostras malignas e não possui valores ausentes na classe. As características dos dados podem ser observadas na tabela 2, na sequência.

Tabela 2 – Atributos da base da dados "*Breast Cancer Wisconsin (Diagnostic)*".

Nome da Variável	Tradução / Significado	Tipo	Valores Ausentes (S/N)
ID	Número Identificador	Identificador	Não
Diagnosis	Diagnóstico	Categórico Nominal Binário	Não
radius1	Média do Raio	Numérico Contínuo	Não
texture1	Média da Textura	Numérico Contínuo	Não
perimeter1	Média do Perímetro	Numérico Contínuo	Não
area1	Média da Área	Numérico Contínuo	Não
smoothness1	Média da Suavidade	Numérico Contínuo	Não
compactness1	Média da Compacidade	Numérico Contínuo	Não
concavity1	Média da Concavidade	Numérico Contínuo	Não
concave_points1	Média dos Pontos Côncavos	Numérico Contínuo	Não
symmetry1	Média da Simetria	Numérico Contínuo	Não
fractal_dimension1	Média da Dimensão Fractal	Numérico Contínuo	Não
radius2	Erro Padrão do Raio	Numérico Contínuo	Não
texture2	Erro Padrão da Textura	Numérico Contínuo	Não
perimeter2	Erro Padrão do Perímetro	Numérico Contínuo	Não
area2	Erro Padrão da Área	Numérico Contínuo	Não
smoothness2	Erro Padrão da Suavidade	Numérico Contínuo	Não
compactness2	Erro Padrão da Compacidade	Numérico Contínuo	Não
concavity2	Erro Padrão da Concavidade	Numérico Contínuo	Não
concave_points2	Erro Padrão dos Pontos Côncavos	Numérico Contínuo	Não
symmetry2	Erro Padrão da Simetria	Numérico Contínuo	Não
fractal_dimension2	Erro Padrão da Dimensão Fractal	Numérico Contínuo	Não
radius3	Pior Raio medido	Numérico Contínuo	Não
texture3	Pior Textura medida	Numérico Contínuo	Não
perimeter3	Pior Perímetro medido	Numérico Contínuo	Não
area3	Pior Área medida	Numérico Contínuo	Não
smoothness3	Pior Suavidade medida	Numérico Contínuo	Não
compactness3	Pior Compacidade medida	Numérico Contínuo	Não
concavity3	Pior Concavidade medida	Numérico Contínuo	Não
concave_points3	Pior Pontos Côncavos medido	Numérico Contínuo	Não
symmetry3	Pior Simetria medida	Numérico Contínuo	Não
fractal_dimension3	Pior Dimensão Frac. medida	Numérico Contínuo	Não

3. Metodologia

Esta seção detalha os procedimentos metodológicos adotados para a execução dos experimentos, abrangendo desde o tratamento dos dados até as métricas escolhidas para a avaliação dos modelos.

3.1. Ferramentas Utilizadas

Os experimentos foram implementados utilizando a linguagem de programação *Python*, executada em ambiente de nuvem *Google Colab*. A principal biblioteca utilizada para a construção e validação dos modelos foi a Scikit-Learn (*sklearn*), apoiada pela biblioteca *Pandas* para manipulação dos dados tabulares, *NumPy* para operações matemáticas e *Matplotlib* em conjunto com o *Seaborn* para criar gráficos e visualizações estatísticas mais atraentes e sofisticadas. Os códigos completos utilizados podem ser observados nos Anexos I, II e III.

3.2. Pré-processamento dos Dados

Para garantir a eficácia dos algoritmos de classificação e evitar resultados enviesados, foram aplicadas técnicas distintas de pré-processamento para cada base de dados, respeitando suas características intrínsecas.

3.2.1. Base Titanic

A base Titanic apresentava desafios relacionados a dados faltantes e variáveis categorias não numéricas. As seguintes etapas foram realizadas:

- **Seleção de Atributos**: Foram removidas as colunas *Name*, *Ticket* e *Cabin*, pois continham alta cardinalidade ou muitos dados ausentes, focando-se nas variáveis com maior potencial preditivo direto (*Pclass, Sex, Age, SibSp, Parch, Fare, Embarked*).
- **Imputação de Valores Ausentes:** Identificou-se a ausência de valores na variável *Age*. Para não descartar registros importantes, optou-se pela imputação utilizando a mediana dos valores, uma medida de tendência central robusta a outliers.
- **Codificação de Variáveis Categóricas:** Os algoritmos de aprendizado de máquina requerem entradas numéricas. A variável *Sex* foi convertida para binário (0 e 1). Para a variável *Embarked* (porto de embarque), que não possui ordem intrínseca, utilizamos a técnica de *One-Hot Encoding*, desmembrando a coluna original em variáveis binárias para evitar que o modelo inferisse uma ordem inexistente entre os portos.

Na sequência, tabela 3, são apresentados os trechos de códigos na linguagem de programação *Python* que foram responsáveis por realizar o pré-processamento supracitado.

Tabela 3 - Códigos para realizar o pré-processamento Titanic.

Trecho do Código	Função
`colunas_uteis = ['Survived', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']`	Variável com o nome das colunas consideradas úteis.
`df_titanic = df_titanic[colunas_uteis]`	*df_titanic* fica apenas com as colunas úteis.
`df_titanic['Age'].fillna(df_titanic['Age'].median())`	Aqui a forma que foi calculado a mediana para completar os valores ausentes com o comando *"fillna"*.
`df_titanic['Sex'].map({'male': 0, 'female': 1})`	Conversão da coluna *Sex* para binário
`pd.get_dummies(df_titanic, columns=['Embarked'], drop_first=True)`	Conversão com a técnica de One-Hot Encoding da coluna *Embarked*

3.2.2. Base Breast Cancer Wisconsin

Esta base não apresentava valores ausentes, porém seus atributos possuíam escalas de grandeza muito discrepantes, como o atributo *Area* que apresenta valores na casa dos milhares enquanto *Smoothness* conta com valores decimais. Além da remoção da coluna 'id' que é um identificador único para cada amostra, foram utilizadas as seguintes abordagens para o pré-processamento desses dados:

- **Normalização**: Para evitar que atributos com valores numéricos maiores dominassem o cálculo da função objetivo dos classificados, foi aplicada a técnica de Padronização. Todos os atributos preditores foram transformados para possuírem média 0 e desvio padrão 1.
- **Transformação da Classe Alvo**: A variável alvo *Diagnosis* (Maligno/Benigno) foi convertida para formato numérico binário (1 para Maligno, 0 para Benigno), devido à classe do tipo Maligno ser a minoria.

Assim pode-se observar na tabela 4, os códigos que foram aplicados na base de dados para fazer o pré-processamento.

Tabela 4 - Código para realizar o pré-processamento na base Breast Cancer Wisconsin.

Trecho do Código	Função
`df_cancer.drop('id', axis=1)`	Remoção da coluna ID, pois ela não serve para análise dos dados.

`df_cancer['diagnosis'].map({'M': 1, 'B': 0})`	Conversão da coluna diagnóstico para número (M=1, B=0).
`scaler = StandardScaler()` `scaler.fit_transform(X_cancer)`	Padronização das características (Normalização).

3.3. Algoritmos Utilizados

Foram selecionados três algoritmos de aprendizagem supervisionada para o estudo comparativo:

1. ***K-Nearest Neighbors* (KNN):** Algoritmo baseado em instância ("preguiçoso") que não constrói um modelo explícito, mas classifica novos dados com base na votação majoritária dos 'k' vizinhos mais próximos no espaço de características.
2. **Árvore de Decisão (*Decision Tree*):** Modelo que utiliza uma estrutura de árvore para criar regras de decisão "se-então", dividindo os dados em nós sucessivos até atingir uma decisão final (folha).
3. ***Random Forest* (Floresta Aleatória):** Este algoritmo é um método de *Ensemble Learning* (aprendizado em conjunto) do tipo *Bagging*, que combina múltiplas árvores de decisão para obter uma predição mais robusta.
    - **Descrição do Funcionamento:** Diferente de uma única árvore, o *Random Forest* constrói diversas árvores de decisão no momento do treinamento.
    - **Passos do Algoritmo:** Primeiro, ele cria várias subamostras do conjunto de dados original (técnica de *Bootstrap*). Em seguida, treina uma árvore para cada amostra. Um diferencial importante é que, em cada divisão de nó, o algoritmo considera apenas um subconjunto aleatório de recursos, aumentando a diversidade. A classificação final é dada pela média (em regressão) ou moda (em classificação) das previsões de todas as árvores individuais, reduzindo a tendência ao *overfitting*.

3.4. Estratégia de Divisão da Base

Foi adotada a estratégia de validação cruzada simples (*holdout*). Os dados foram divididos aleatoriamente em dois subconjuntos disjuntos:

- **Treinamento:** 70% dos dados, utilizados para o ajuste dos parâmetros do modelo.

- **Teste:** 30% dos dados, utilizados exclusivamente para a avaliação final e cálculo das métricas.

Para garantir a reprodutibilidade dos experimentos, definiu-se uma semente aleatória com valor fixo (random_state=42) ao utilizar a função "*train_test_split*" para realizar o *holdout*.

3.5. Medidas de Avaliação

Para quantificar a eficácia dos modelos, utilizou-se a Matriz de Confusão para extrair as frequências de acertos e erros. A principal métrica calculada foi a Acurácia, dada pela fórmula:

$$Acurácia = \frac{VP + VN}{VP + VN + FP + FN}$$

Onde:

- VP: Verdadeiros Positivos (Acertos da classe positiva).
- VN: Verdadeiros Negativos (Acertos da classe negativa).
- FP: Falsos Positivos (Erro Tipo I).
- FN: Falsos Negativos (Erro Tipo II).

Além da Acurácia, foi utilizada a métrica de Sensibilidade (ou *Recall*). Esta medida é fundamental no contexto de diagnósticos médicos (como a base *Breast Cancer*), pois avalia a capacidade do modelo de identificar corretamente os casos positivos reais (tumores malignos), penalizando os Falsos Negativos (erros do Tipo II). A fórmula utilizada é:

$$Sensibilidade = \frac{VP}{VP + FN}$$

Onde:

- VP: Verdadeiros Positivos (Acertos da classe positiva).
- FN: Falsos Negativos (Erro Tipo II).

3.6. Configuração dos Algoritmos

Os hiper parâmetros utilizados para configurar os algoritmos nos experimentos foram:

- **KNN:** n_neighbors=3, 5 e 7 (considerando os 3, 5 e 7 vizinhos mais próximos).

- **Árvore de Decisão:** max_depth=3 (profundidade máxima limitada a 3 níveis para facilitar a interpretabilidade).
- ***Random Forest*:** n_estimators=100 (criação de 100 árvores independentes) e random_state=42 para reprodutibilidade.

Para cumprir a etapa de validação e garantir a melhor generalização, foram testadas parametrizações alternativas nos algoritmos. Para a Árvore de Decisão, testou-se a profundidade max_depth = 5, e para o Random Forest, testou-se um conjunto menor com n_estimators = 50. Após a avaliação das predições, as configurações iniciais (max_depth = 3 e n_estimators = 100) foram mantidas como as principais para a discussão, pois apresentaram o melhor equilíbrio entre interpretabilidade e prevenção de *overfitting*.

4. Discussão e resultados

Nesta seção são apresentados e discutidos os resultados obtidos a partir da aplicação dos algoritmos *K-Nearest Neighbors* (KNN), Árvore de Decisão (*Decision Tree*) e *Random Forest*. A avaliação foi realizada utilizando a estratégia de validação cruzada simples, com divisão de 70% dos dados para treinamento e 30% para testes, visando garantir a generalização dos modelos.

Após a realização do pré-processamento nos dados *Titanic*, foi montada uma tabela, Tabela 5, com as primeiras cinco linhas do conjunto dos dados, para que esses dados pudessem ser observados após a etapa de tratamento inicial.

Tabela 5 – Dados parciais da base de dados Titanic após pré-processamento

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S	Embarked_C
3	0	22.0	1	0	7.2500	0	1	0
1	1	38.0	1	0	71.2833	0	0	0
3	1	26.0	0	0	7.9250	0	1	0
1	1	35.0	1	0	53.1000	0	1	0
3	0	35.0	0	0	8.0500	0	1	0

Fonte: Elaborado pelos autores (2026), com base em dados do Titanic - Dataset" (ALI, [s.d.]).

E quando o pré-processamento foi realizado na base de dados *Breast Cancer Wisconsin* os dados normalizados foram obtidos, sendo que as primeiras 5 amostras podem ser observadas na tabela 6 abaixo. Nela é possível verificar como ficou a normalização de parte dos 30 atributos dessa base de dados.

Tabela 6 – Dados parciais da base de dados *Breast Cancer Wisconsin* após o pré-processamento.					
Variáveis	**Amostra 1**	**Amostra 2**	**Amostra 3**	**Amostra 4**	**Amostra 5**
radius_mean	1.097064	1.829821	1.579888	-0.768909	1.750297
texture_mean	-2.073335	-0.353632	0.456187	0.253732	-1.151816
perimeter_mean	1.269934	1.685955	1.566503	-0.592687	1.776573
area_mean	0.984375	1.908708	1.558884	-0.764464	1.826229
smoothness_mean	1.568466	-0.826962	0.942210	3.283553	0.280372
compactness_mean	3.283515	-0.487072	1.052926	3.402909	0.539340
concavity_mean	2.652874	-0.023846	1.363478	1.915897	1.371011
concave_points_mean	2.532475	0.548144	2.037231	1.451707	1.428493
symmetry_mean	2.217515	0.001392	0.939685	2.867383	-0.009560
(...)	(...)	(...)	(...)	(...)	(...)
diagnosis	1	1	1	1	1

Fonte: Elaborado pelos autores (2026), com base em dados do Titanic - Dataset" (ALI, [s.d.]).

4.1. Análise do Cenário 1: Titanic

Para o problema de classificação binária dos passageiros do Titanic (sobreviventes *versus* não sobreviventes), observou-se uma variação significativa de desempenho entre os algoritmos testados.

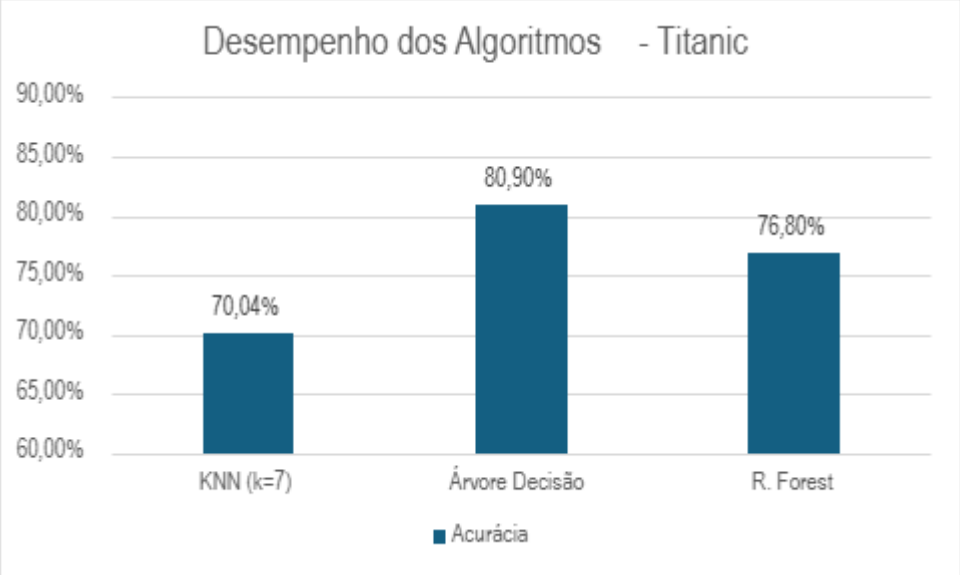
Inicialmente três distâncias distintas para vizinhos (k) para o modelo KNN foram testadas ("k=3", "k=5" e "k=7"), para verificar qual apresentava melhor performance. O "k" que obteve melhor resultado global foi o igual a "7" alcançando a acurácia de 70,04%, conforme apresentado na tabela 7. Portanto, o KNN com k=7 será o escolhido para esse algoritmo.

Tabela 7 – Valores de K e suas respectivas acurácias	
KNN - Valor de K	**Acurácia**
3	68,91%
5	69,29%
7	70,04%

Neste cenário, conforme pode ser observado no gráfico 1, o algoritmo Árvore de Decisão obteve o melhor desempenho geral, com uma acurácia de 80,90%. Surpreendentemente, esse modelo simples superou o *Random Forest*, que apresentou uma acurácia de 76,78%. Isso indica que, para esse conjunto de dados específico, regras de decisão diretas foram adequadas para identificar o padrão de sobrevivência sem precisar da

complexidade de um *ensemble*. O modelo *KNN* (k=7) apresentou a menor taxa de acerto (70,04%), o que sugere problemas ao lidar com a dimensionalidade e a natureza mista dos dados.

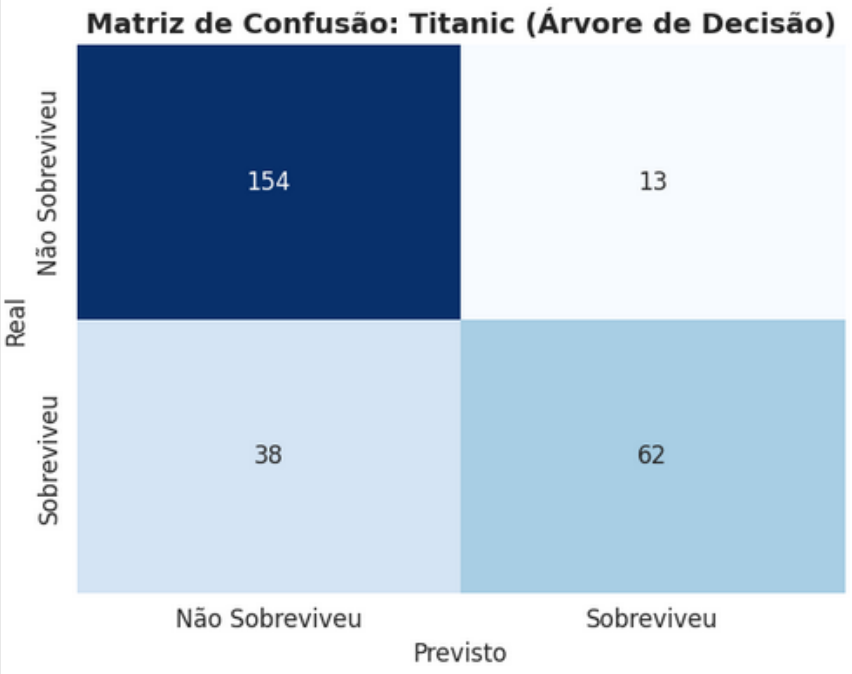
Gráfico 1 - Desempenho dos Algoritmos de classificação para a base de dados Titanic.



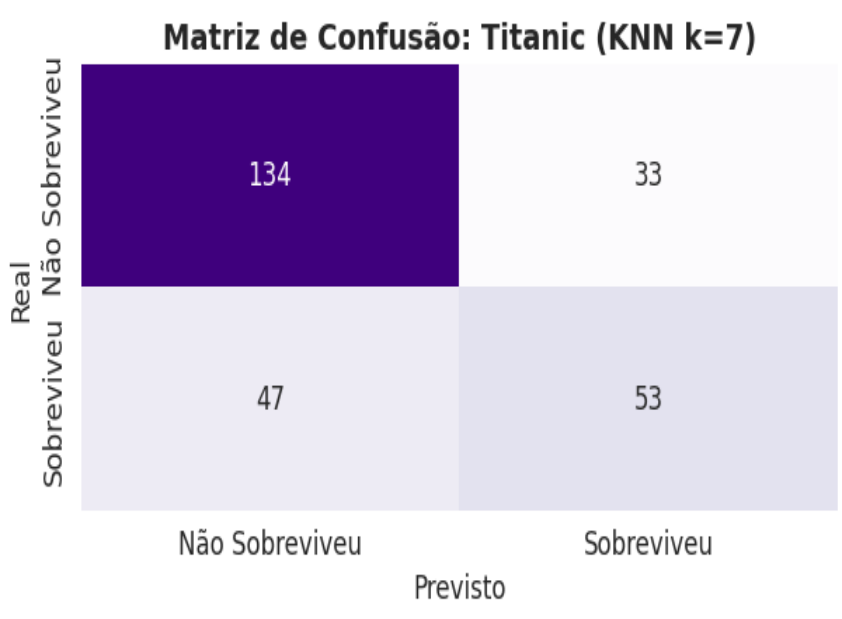
A análise detalhada das matrizes de confusão e da importância dos atributos revelou comportamentos distintos:

- **Falsos Negativos (Erro Tipo II):** A Árvore de Decisão apresentou um equilíbrio melhor, errando em 38 casos de sobreviventes não detectados (tabela 8[1]), enquanto o *KNN* falhou em 47 casos (tabela 8[2]).
- **Interpretabilidade:** A Árvore de Decisão baseou sua classificação majoritariamente na variável *Sex* (Gênero), atribuindo-lhe 63,1% de importância, seguida pela classe social (*Pclass*, 15,8%). Já o *Random Forest* distribuiu a importância de forma mais equitativa entre Tarifa (28,0%), Sexo (25,3%) e Idade (24,8%). Esses resultados corroboram estatisticamente com protocolo de salvamento histórico de "mulheres e crianças primeiro", capturada perfeitamente pela Árvore de Decisão.

Tabela 8 – Matriz de Confusão para os algoritmos da base de dados Titanic.



[1] Matriz de Confusão para o algoritmo Árvore de Decisão (Titanic).



[2] Matriz de Confusão para o algoritmo KNN com k=7 (Titanic).

4.2. Análise do Cenário 2: *Breast Cancer Wisconsin*

Neste cenário, a análise crítica dos resultados priorizou a minimização dos Falsos Negativos (classificar um tumor maligno como benigno), dado que, no contexto médico, este erro pode atrasar o tratamento e comprometer a vida do paciente.

Para o modelo KNN, adotou-se o valor k=3, visando a otimização dos recursos computacionais. Tal decisão fundamenta-se nos testes realizados com as variações de $k \in \{3,5,7\}$, nos quais se observou equivalência nos índices de acurácia, conforme apresentado na Tabela 9.

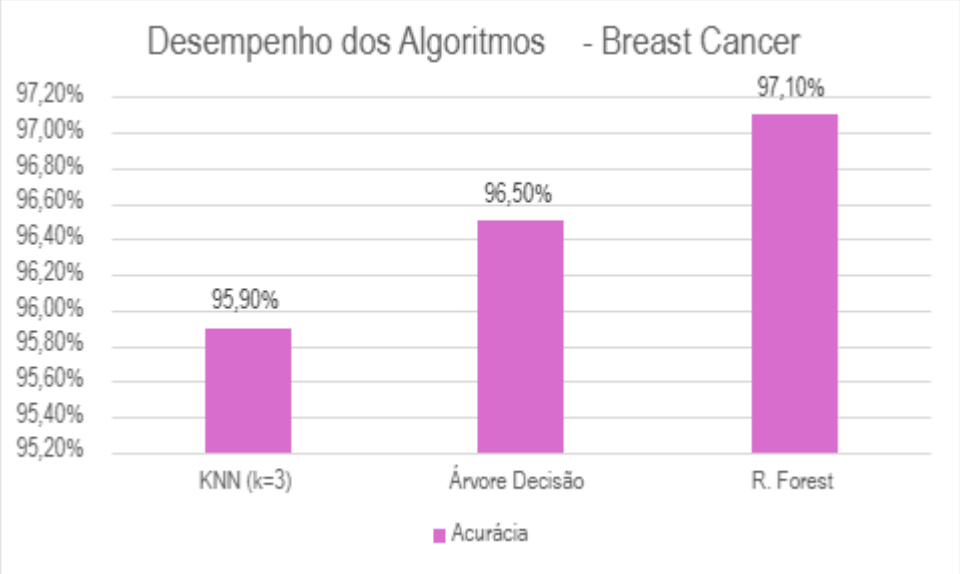
Tabela 9 – Valores de K e suas respectivas acurácias para a base de dados Breast Cancer Wisconsin

KNN - Valor de K	Acurácia
3	95,91%
5	95,91%
7	95,91%

Os resultados de acurácia global obtidos foram muito próximos, podendo ser observados no gráfico 2, evidenciando a alta separabilidade linear dos dados:

- *Random Forest*: 97,08% (Melhor desempenho).
- Árvore de Decisão: 96,49%.
- *KNN* (k=3): 95,91%.

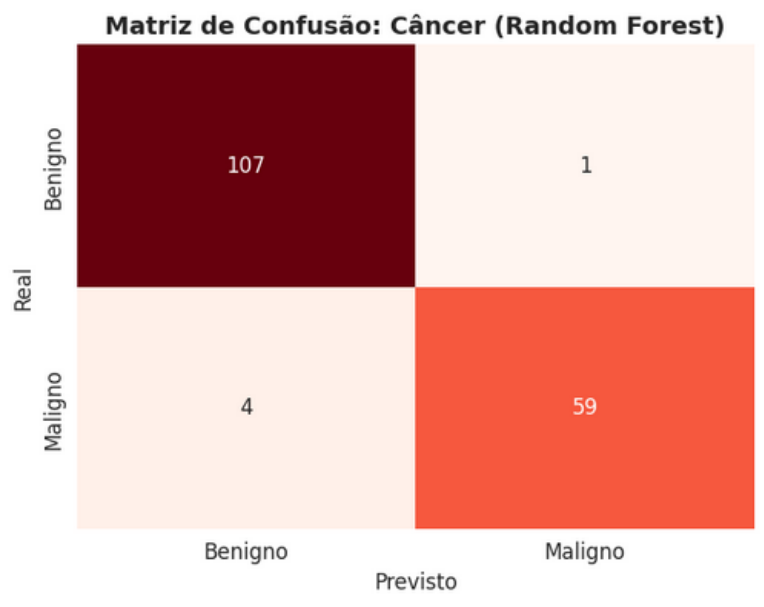
Gráfico 2 - Desempenho dos Algoritmos de classificação para a base de dados *Breast Cancer*.



Um ponto crítico observado na Matriz de Confusão é que todos os três algoritmos apresentaram exatamente 4 Falsos Negativos. Ou seja, independente da complexidade do modelo, 4 casos malignos não foram detectados. A superioridade do *Random Forest* se deu

pela redução dos Falsos Positivos (apenas 1 erro), tabela 10, tornando-o o modelo mais preciso globalmente.

Tabela 10 – Matriz de Confusão para o algoritmo Randon Forest (Breast Cancer).



Em relação aos atributos, a variável *concave_points_mean* (média de pontos côncavos) foi determinante. Na Árvore de Decisão, ela representou sozinha 81,0% da decisão, enquanto no *Random Forest* ela também liderou (14,2%), seguida por *concave_points_worst* (12,7%), indicando que a geometria celular é o preditor mais forte para a patologia.

4.3. Comparativo Geral

Para consolidar essa análise e facilitar a visualização dos resultados absolutos, a Tabela 11 sumariza o desempenho final e as configurações principais adotadas para cada algoritmo em ambas as bases de dados avaliadas.

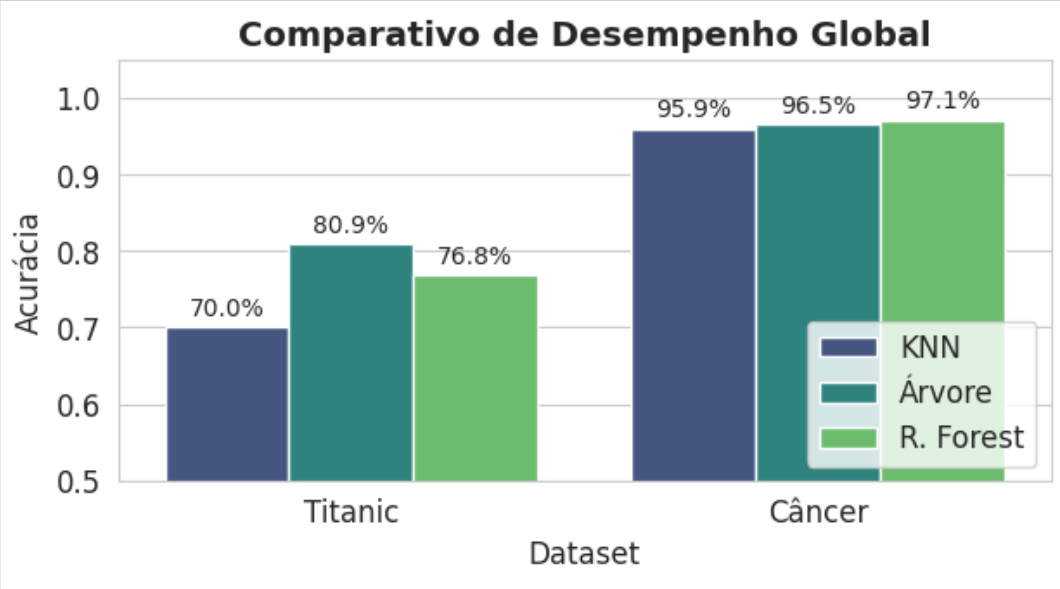
Tabela 11 – Sumarização do Desempenho dos Algoritmos

Base de Dados	Algoritmo	Configuração Principal	Acurácia
Titanic	Árvore de Decisão	max_depth = 3	80,90%
Titanic	Random Forest	n_estimators = 100	76,80%
Titanic	KNN	k = 7	70,04%
Breast Cancer	Random Forest	n_estimators = 100	97,08%
Breast Cancer	Árvore de Decisão	max_depth = 3	96,49%
Breast Cancer	KNN	k = 3	95,91%

Ao comparar as duas abordagens, observa-se (pelo gráfico 3) que no conjunto de dados do *Titanic*, que possui dados categóricos e valores ausentes tratados, a Árvore de Decisão

simples superou modelos mais complexos. Já no *dataset* de Câncer, caracterizado por atributos numéricos contínuos de alta precisão, o desempenho dos modelos tendeu ao equilíbrio, com leve vantagem para métodos de *ensemble* como o *Random Forest*.

Gráfico 3 – Comparativo de Desempenho Global (*Titanic* x *Cancer Breast*)



Conclui-se que, para o problema médico, embora a acurácia seja alta (>97%), a persistência de falsos negativos sugere a necessidade de ajuste no limiar de decisão ou inclusão de mais dados para garantir segurança clínica total.

5. Uso de LLMs

Para o desenvolvimento dos códigos e scripts em *Python* apresentados neste trabalho, utilizou-se como ferramenta de apoio o modelo de linguagem *Gemini* (*Google*). A ferramenta atuou como um assistente de programação, auxiliando na sintaxe das bibliotecas *Scikit-Learn* e *Pandas*, bem como na formatação da saída dos resultados para facilitar a interpretação.

Cabe ressaltar que todo o código gerado foi revisado, testado e validado pelos autores do trabalho para garantir a corretude técnica e a adequação aos objetivos da disciplina. Abaixo, listam-se os principais *prompts* (comandos) utilizados em cada etapa do desenvolvimento:

5.1. *Prompts* para Pré-processamento

Para a limpeza e preparação das bases de dados, foram utilizados comandos focados no tratamento de valores nulos e normalização:

> *Gere um código Python utilizando Pandas para pré-processar o dataset do Titanic. O código deve preencher as idades faltantes com a mediana, o porto de embarque com a moda, e*

converter as variáveis categóricas 'Sex' e 'Embarked' para valores numéricos. Remova colunas que não influenciam na sobrevivência, como nomes e tickets.

Para a base Breast Cancer Wisconsin, crie um script que carregue os dados, converta a variável alvo 'Diagnosis' (M/B) para binário (0 e 1) e aplique o StandardScaler nos atributos numéricos para normalizar a escala dos dados.

5.2. Prompts para Execução dos Algoritmos

Para a construção dos modelos, solicitou-se a implementação comparativa dos três algoritmos exigidos:

Escreva um código que divida a base de dados em treino (70%) e teste (30%) usando uma semente fixa (random_state=42). Em seguida, instancie e treine três classificadores: KNN (com k=3), Árvore de Decisão (profundidade máxima 3) e Random Forest (com 100 árvores). Armazene os modelos treinados para avaliação posterior.

6. Conclusão

O presente trabalho cumpriu o seu objetivo de analisar e comparar o desempenho de três algoritmos de aprendizagem de máquina supervisionada: *K-Nearest Neighbors* (KNN), Árvore de Decisão e *Random Forest*, aplicados a dois cenários distintos: a predição de sobrevivência no *Titanic* e o diagnóstico de câncer de mama.

Os experimentos demonstraram que não existe um algoritmo universalmente superior; a escolha ideal depende da estrutura dos dados e do problema de negócio. No caso do *Titanic*, caracterizado por dados mistos e regras sociais bem definidas (como a prioridade de salvar mulheres e crianças em protocolos de urgência), a Árvore de Decisão simples mostrou-se mais eficiente e interpretável do que modelos complexos, superando inclusive o *Random Forest* em acurácia. Isso evidencia que, em certos contextos, a simplicidade do modelo facilita a captura de padrões determinísticos.

Por outro lado, no *dataset Breast Cancer*, que possui atributos numéricos contínuos e alta dimensionalidade, os modelos de *ensemble*, como o *Random Forest*, apresentaram maior robustez. No entanto, a análise crítica revelou um desafio persistente: todos os algoritmos falharam em detectar uma pequena parcela de casos malignos (Falsos Negativos). Esse achado ressalta que, para aplicações críticas em saúde, a alta acurácia isolada não é suficiente, sendo necessário ajustar os modelos para priorizar a sensibilidade.

Conclui-se, portanto, que o sucesso de um projeto de Ciência de Dados reside na combinação equilibrada entre um pré-processamento adequado (limpeza e normalização), a

escolha correta do algoritmo e, fundamentalmente, a interpretação humana das métricas para garantir que o modelo atenda às necessidades do mundo real.

7. Referências Bibliográficas

ALI, Syed Hamza. **Titanic-Dataset (train.csv)**. [S. l.]: Kaggle, [s.d.]. Disponível em: <https://www.kaggle.com/datasets/hesh97/titanicdataset-traincsv?select=train.csv>. Acesso em: 9 fev. 2026.

CAVIQUE, Luís. Big data e data science. Boletim da APDIO, p. 11-14, 2014.

RMS Titanic. In: WIKIPÉDIA: a enciclopédia livre. [S.I.]: Wikimedia Foundation, 2025. Disponível em: <https://pt.wikipedia.org/wiki/RMS_Titanic>. Acesso em: 3 fev. 2026.

WOLBERG, William; STREET, Nick; MANGASARIAN, Olvi. Breast Cancer Wisconsin (Diagnostic). [S. l.]: UCI Machine Learning Repository, 1995. DOI: 10.24432/C5DW2B. Disponível em: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>. Acesso em: 3 fev. 2026.

ANEXO I – Códigos Colab (Pré-processamento)

Pré-processamento Titanic

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder

print("--- Processando Titanic ---")

# Carrega os dados
df_titanic = pd.read_csv('train.csv')

# Seleciona as colunas úteis
colunas_uteis = ['Survived', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
df_titanic = df_titanic[colunas_uteis]

# Preenche valores vazios com a mediana
df_titanic['Age'] = df_titanic['Age'].fillna(df_titanic['Age'].median())
df_titanic['Fare'] = df_titanic['Fare'].fillna(df_titanic['Fare'].median())

# Remove valores vazios na coluna de embarque
df_titanic = df_titanic.dropna(subset=['Embarked'])

# Converte texto para número (Sexo)
df_titanic['Sex'] = df_titanic['Sex'].map({'male': 0, 'female': 1})

# Aplica One-Hot Encoding (Embarque)
df_titanic = pd.get_dummies(df_titanic, columns=['Embarked'], drop_first=True)

# Exibe o resultado
print("Titanic processado (primeiras 5 linhas):")
display(df_titanic.head())
print("\n")

# Salva o arquivo limpo
df_titanic.to_csv('titanic_processado.csv', index=False)
print("\nArquivo 'titanic_processado.csv' salvo com sucesso!")
```

Pré-processamento Breast Cancer

```python
print("--- Processando Breast Cancer ---")

# Define os nomes das colunas
colunas_cancer = ['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
                  'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
                  'concave_points_mean', 'symmetry_mean', 'fractal_dimension_mean',
                  'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
                  'compactness_se', 'concavity_se', 'concave_points_se', 'symmetry_se',
                  'fractal_dimension_se', 'radius_worst', 'texture_worst',
                  'perimeter_worst', 'area_worst', 'smoothness_worst',
                  'compactness_worst', 'concavity_worst', 'concave_points_worst',
                  'symmetry_worst', 'fractal_dimension_worst']

# Carrega os dados
df_cancer = pd.read_csv('wdbc.data', header=None, names=colunas_cancer)

# Remove a coluna ID
df_cancer = df_cancer.drop('id', axis=1)

# Converte o diagnóstico para número (M=1, B=0)
df_cancer['diagnosis'] = df_cancer['diagnosis'].map({'M': 1, 'B': 0})

# Separa características (X) e alvo (y)
X_cancer = df_cancer.drop('diagnosis', axis=1)
y_cancer = df_cancer['diagnosis']

# Normaliza os dados
scaler = StandardScaler()
X_cancer_scaled = scaler.fit_transform(X_cancer)

# Junta os dados normalizados com o alvo
df_cancer_final = pd.DataFrame(X_cancer_scaled, columns=X_cancer.columns)
df_cancer_final['diagnosis'] = y_cancer.values

# Exibe o resultado
print("Breast Cancer processado e normalizado (primeiras 5 linhas):")
display(df_cancer_final.head())

# Salva o arquivo limpo
df_cancer_final.to_csv('cancer_processado.csv', index=False)
print("\nArquivo 'cancer_processado.csv' salvo com sucesso!")
```

ANEXO II – Códigos Colab (Classificação)

Rodando os algoritmos

```python
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, recall_score # Adicionado o recall_score
import pandas as pd

# Treina o modelo e calcula as métricas
def avaliar_modelo(nome_modelo, modelo, X_train, X_test, y_train, y_test):
    modelo.fit(X_train, y_train)
    y_pred = modelo.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='weighted')
    # Adicionando o cálculo do Recall (Sensibilidade)
    recall = recall_score(y_test, y_pred, average='weighted')
    return acc, f1, recall

# Roda os algoritmos e exibe a tabela de resultados
def rodar_experimentos(X, y, nome_dataset):
    print(f"\n--- Resultados para: {nome_dataset} ---")

    # Separa 70% para treino e 30% para teste
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    # Define os modelos e seus parâmetros (Ajustados para 5 e 50)
    configuracoes = [
        ("KNN (k=3)", KNeighborsClassifier(n_neighbors=3)),
        ("KNN (k=5)", KNeighborsClassifier(n_neighbors=5)),
        ("KNN (k=7)", KNeighborsClassifier(n_neighbors=7)),
        ("Árvore (profundidade=3)", DecisionTreeClassifier(max_depth=3, random_state=42)),
        ("Árvore (profundidade=5)", DecisionTreeClassifier(max_depth=5, random_state=42)), # Novo teste
        ("Random Forest (50 árvores)", RandomForestClassifier(n_estimators=50, random_state=42)), # Novo teste
        ("Random Forest (100 árvores)", RandomForestClassifier(n_estimators=100, random_state=42))
    ]

    resultados = []

    # Avalia cada modelo
    for nome, modelo in configuracoes:
        acc, f1, recall = avaliar_modelo(nome, modelo, X_train, X_test, y_train, y_test)
        # Adicionando o Recall na tabela final
        resultados.append({'Algoritmo': nome, 'Acurácia': f"{acc:.4f}", 'F1-Score': f"{f1:.4f}", 'Recall': f"{recall:.4f}"})

    # Exibe os resultados formatados
    df_res = pd.DataFrame(resultados)
    display(df_res)

# Executa as análises para o Titanic
if 'df_titanic' in locals():
    X_titanic = df_titanic.drop('Survived', axis=1)
    y_titanic = df_titanic['Survived']
    rodar_experimentos(X_titanic, y_titanic, "TITANIC")

# Executa as análises para o Breast Cancer
if 'df_cancer_final' in locals():
    X_cancer = df_cancer_final.drop('diagnosis', axis=1)
    y_cancer = df_cancer_final['diagnosis']
    rodar_experimentos(X_cancer, y_cancer, "BREAST CANCER")
```

ANEXO III – Códigos Colab (Gráficos)

Gráficos

```python
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
from sklearn.metrics import confusion_matrix

# Configura o visual base dos gráficos
sns.set_style("whitegrid")
plt.rcParams.update({'font.size': 12})

# Dados de acurácia dos modelos testados
dados_resultados = {
    'Algoritmo': ['KNN', 'Árvore', 'R. Forest', 'KNN', 'Árvore', 'R. Forest'],
    'Dataset': ['Titanic', 'Titanic', 'Titanic', 'Câncer', 'Câncer', 'Câncer'],
    'Acurácia': [0.7004, 0.8090, 0.7678, 0.9591, 0.9649, 0.9708]
}
df_res = pd.DataFrame(dados_resultados)

# Cria a tela para os gráficos (3 linhas x 2 colunas)
fig, axes = plt.subplots(3, 2, figsize=(16, 12))
plt.subplots_adjust(hspace=0.4, wspace=0.3)

# 1. Gráfico de Barras: Comparativo geral de acurácia
sns.barplot(data=df_res, x='Dataset', y='Acurácia', hue='Algoritmo', ax=axes[0, 0], palette='viridis')
axes[0, 0].set_title('Comparativo de Desempenho Global', fontsize=14, fontweight='bold')
axes[0, 0].set_ylim(0.5, 1.05)
axes[0, 0].legend(loc='lower right')

# Adiciona as porcentagens no topo de cada barra
for p in axes[0, 0].patches:
    axes[0, 0].annotate(f'{p.get_height():.1%}',
                        (p.get_x() + p.get_width() / 2., p.get_height()),
                        ha='center', va='center', xytext=(0, 8), textcoords='offset points', fontsize=10)

# 2. Matriz de Confusão: Titanic (Árvore de Decisão)
cm_tit = np.array([[154, 13], [38, 62]])
sns.heatmap(cm_tit, annot=True, fmt='d', cmap='Blues', ax=axes[0, 1], cbar=False,
            xticklabels=['Não Sobreviveu', 'Sobreviveu'],
            yticklabels=['Não Sobreviveu', 'Sobreviveu'])
axes[0, 1].set_title('Matriz de Confusão: Titanic (Árvore de Decisão)', fontsize=14, fontweight='bold')
axes[0, 1].set_ylabel('Real')
axes[0, 1].set_xlabel('Previsto')

# 3. Matriz de Confusão: Titanic (KNN k=7)
knn_tit = np.array([[134, 33], [47, 53]])
sns.heatmap(knn_tit, annot=True, fmt='d', cmap='Purples', ax=axes[2, 0], cbar=False,
            xticklabels=['Não Sobreviveu', 'Sobreviveu'],
            yticklabels=['Não Sobreviveu', 'Sobreviveu'])
axes[2, 0].set_title('Matriz de Confusão: Titanic (KNN k=7)', fontsize=14, fontweight='bold')
axes[2, 0].set_ylabel('Real')
axes[2, 0].set_xlabel('Previsto')

# 4. Matriz de Confusão: Câncer (Random Forest)
cm_can = np.array([[107, 1], [4, 59]])
sns.heatmap(cm_can, annot=True, fmt='d', cmap='Reds', ax=axes[1, 0], cbar=False,
            xticklabels=['Benigno', 'Maligno'],
            yticklabels=['Benigno', 'Maligno'])
axes[1, 0].set_title('Matriz de Confusão: Câncer (Random Forest)', fontsize=14, fontweight='bold')
axes[1, 0].set_ylabel('Real')
axes[1, 0].set_xlabel('Previsto')

# 5. Gráfico de Barras: Importância das variáveis (Titanic)
feat_imp = pd.Series({'Sex': 0.6305, 'Pclass': 0.1585, 'Age': 0.0572, 'Fare': 0.0556, 'Embarked': 0.0560})
feat_imp = feat_imp.sort_values(ascending=True)
feat_imp.plot(kind='barh', ax=axes[1, 1], color='#4c72b0')
axes[1, 1].set_title('O que definiu a sobrevivência? (Top 5)', fontsize=14, fontweight='bold')
axes[1, 1].set_xlabel('Importância Relativa')

plt.show()
```